Curatio

How the Server and the BioBERT Model Work Together

A plain-English guide to the API layer, the ML service, and the fine-tuned model.

Final Year Project

June 6, 2026

This document explains the **runtime architecture**: what happens when a patient types a chief complaint in the browser, how the Node.js API forwards it to Python, how BioBERT runs inference, and what JSON comes back. Read this before your demo or viva if someone asks "how does the backend actually work?"

Contents

1 The big picture in one minute

Curatio is **not** one monolithic program. At runtime there are three cooperating pieces:

1. **Client** (React/Vite) — the web UI in the browser. It collects text and displays the prediction. It never loads the 414 MB model.
2. **API server** (Node.js/Express on port 5000) — a thin *gateway*: CORS, JSON validation, health checks, and forwarding requests to the ML service.
3. **ML service** (Python/FastAPI on port 8001) — loads BioBERT once at startup, runs tokenisation + inference, maps acuity to SATS colour, and flags low-confidence cases.

THE IDEA

Why split Node and Python?

- BioBERT lives in the **Python/Hugging Face** ecosystem (PyTorch, `transformers`).
- The web stack is **JavaScript/React** — natural for HTTP, CORS, and deployment to Render/Vercel.
- The API is a **proxy**: it keeps the browser simple (one URL) while the heavy ML work stays in a dedicated Python process that can be scaled or redeployed independently.

1.1 Architecture diagram

Browser	⟶	**Express API**	⟶	**FastAPI ML**	⟶	**BioBERT weights**
React SPA	⟶	`curatio/server`	⟶	`curatio/server/ml`	⟶	local dir or HF Hub

Local dev: `localhost:5173` → proxy `/api` → `:5000` → `:8001` → `MODEL_PATH`

Production: Vercel → Render → Hugging Face Space → `MODEL_ID` on the Hub

2 Starting the stack locally

From `curatio/server`, one command starts **both** processes:

```
npm run dev
```

This uses `concurrently` to run:

- **api** — `nodemon index.js` → Express on `http://localhost:5000`
- **ml** — `uvicorn app:app -reload -port 8001` → FastAPI on `http://127.0.0.1:8001`

The client (`curatio/client`) runs separately:

```
npm run dev    # Vite on http://localhost:5173
```

Vite proxies any path starting with `/api` to port 5000, so the browser can call `/api/triage/predict` without CORS issues during development.

3 End-to-end: one prediction request

This is the path when someone submits "central crushing chest pain" on the **Test BioBERT Triage** page.

3.1 Step 1 — Browser sends JSON

Client: TriageTestPage.jsx

```
POST /api/triage/predict
Content-Type: application/json

{ "text": "central crushing chest pain, radiating to left arm" }
```

- Locally, `VITE_API_URL` is empty, so the URL is relative and hits the Vite proxy.
- In production, `VITE_API_URL` is set to the Render API origin (e.g. `https://curatio-api.onrender.com`).

3.2 Step 2 — Express validates and forwards

Server: routes/triage.js

Express receives the body, checks that `text` is a non-empty string, then **does not** run any ML itself. It forwards the same JSON to the Python service:

```
POST ${ML_SERVICE_URL}/predict
{ "text": "central crushing chest pain, radiating to left arm" }
```

`ML_SERVICE_URL` defaults to `http://127.0.0.1:8001` locally; in production it points to your Hugging Face Space URL.

The Express layer also:

- Applies **CORS** — only allows origins listed in `CLIENT_URL` (plus localhost in dev).
- Exposes `GET /api/health` (API alive) and `GET /api/triage/health` (proxies to ML `/health`).
- Returns `503` with a helpful message if Python is not running.

3.3 Step 3 — FastAPI receives and calls `predict()`

ML service: ml/app.py

On startup, the `lifespan` hook calls `load_model()` once — BioBERT is loaded into memory before the first request.

```
@app.post("/predict")
def predict_endpoint(body: PredictRequest):
    return predict(body.text)
```

`PredictRequest` enforces `text` length 1–2000 characters (Pydantic validation).

3.4 Step 4 — Inside `predict()`: what the model actually does

ML service: ml/predict.py

1. **Load pipeline** (singleton) — first call loads tokenizer + model; later calls reuse the in-memory pipeline.
2. **Tokenise** — `pipe(text, truncation=True, max_length=128)`. Only the first 128 tokens are kept (matches training and presentation slides).
3. **Forward pass** — 12 transformer layers produce logits for 5 classes (`LABEL_0` ... `LABEL_4`).

4. **Softmax** — Hugging Face `text-classification` pipeline returns a score (probability) for every class.
5. **Map labels to acuity** — `LABEL_k` → acuity level `k+1` (levels 1–5).
6. **Pick winner** — highest probability = predicted acuity; that value = **confidence**.
7. **SATS mapping** — level 1 → Red, 2 → Orange, 3 → Yellow, 4–5 → Green.
8. **Confidence gate** — if `confidence < CONFIDENCE_THRESHOLD` (default 0.85), set `bayesian_candidate: true`.

THE IDEA

This is **only the NLP layer** from your presentation (Step 4 / BioBERT). The live test page returns BioBERT's language-based triage. Full Curatio fusion (TEWS + Bayesian + safety rules) is the *research design* shown in slides; the deployed test endpoint implements the **fine-tuned classifier** part so you can demo real inference.

3.5 Step 5 — JSON response travels back

Python returns a dict → Express passes it through unchanged → React displays it.

Example response:

```
{
  "text": "central crushing chest pain, radiating to left arm",
  "predicted_acuity_level": 2,
  "predicted_class_index": 1,
  "confidence": 0.94,
  "probabilities": [
    { "level": 2, "class_index": 1, "probability": 0.94 },
    { "level": 3, "class_index": 2, "probability": 0.04 },
    ...
  ],
  "sats_colour": "Orange",
  "bayesian_candidate": false
}
```

- `predicted_acuity_level` — the winning class (1 = most urgent).
- `confidence` — $\max_c \hat{y}_c$ from the presentation (softmax peak).
- `bayesian_candidate` — `true` when confidence < 0.85; in the full system this would trigger the Bayesian fallback at fusion.
- `probabilities` — full distribution for transparency / debugging.

4 Where the model weights live

The ML service resolves the model source at startup:

Environment	Variable	Behaviour
Local dev	`MODEL_PATH`	Load from disk under `fine-tuned-biobert/...`
Production	`MODEL_ID`	Download from Hugging Face Hub at startup
Fallback	(default path)	Same local folder if `MODEL_PATH` unset

`predict.py -- resolve_model_source()`

```
if MODEL_ID is set:
```

```
    source = "your-username/curatio-biobert-triage"   # Hub repo
else:
    source = local MODEL_PATH directory               # model.safetensors on disk
```

Weights are loaded via Hugging Face `AutoTokenizer` and `AutoModelForSequenceClassification.from_pre` The service builds a `text-classification` pipeline with `top_k=None` so **all five** class probabilities are returned (needed for confidence and for demo tables).

Device: CPU by default (`device=-1`); uses CUDA if PyTorch sees a GPU.

5 File map: who owns what

File	Responsibility
`server/index.js`	Express app: CORS, JSON middleware, mounts `/api/triage`, `/api/health`.
`server/routes/triage.js`	Proxy routes: `POST /predict`, `GET /health` → `ML_SERVICE_URL`.
`server/ml/app.py`	FastAPI app: lifespan loads model; HTTP endpoints `/health`, `/predict`.
`server/ml/predict.py`	Core logic: load model, run pipeline, map acuity/SATS, confidence gate.
`server/ml/Dockerfile`	Container for Hugging Face Spaces (uvicorn on port 7860).
`client/vite.config.js`	Dev proxy: `/api` → `localhost:5000`.
`client/.../TriageTestPage.jsx`	UI that calls `/api/triage/predict` and renders probabilities.

6 Health checks and failure modes

6.1 Health endpoints

- `GET /api/health` — Express only: `{ status: "ok", service: "curatio-server" }`.
- `GET /api/triage/health` — Express forwards to ML `/health`.
- `GET /health` (ML) — reports `model_path`, `model_source`, `weights_found`, `model_loaded`.

Use these before a live demo to warm cold-started services (Render free tier, HF Spaces sleep).

6.2 Common errors

HTTP	Cause	Fix
400	Empty `text`	Client validation / user input
503	ML not running	`npm run dev` from `curatio/server`
503	Weights missing	Set `MODEL_PATH` or upload Hub + `MODEL_ID`
CORS error	Wrong origin	Set `CLIENT_URL` on Render to Vercel URL
Slow first request	Cold start	Hit `/health` 1–2 min before demo

7 Production deployment (how the pieces connect)

Deploy order: **Hub model → HF Space → Render API → Vercel client**.

1. Upload `model.safetensors` to Hugging Face Hub → `MODEL_ID`.
2. Deploy `server/ml/` as a Docker Space → `ML_SERVICE_URL`.
3. Deploy `server/` to Render with `ML_SERVICE_URL` and `CLIENT_URL`.
4. Deploy `client/` to Vercel with `VITE_API_URL` = Render URL.

Full step-by-step commands live in `curatio/DEPLOYMENT.md`.

THE IDEA

In production the browser **never** talks to Python directly. It only sees the Render API. That keeps the ML service private, lets you rotate models without redeploying the client, and matches standard microservice practice.

8 How this connects to your presentation maths

Presentation concept	Where it happens in code
Token limit 128	`predict.py`: `max_length=128`, `truncation=True`
12 encoder layers	Inside loaded BioBERT weights (not reimplemented in app code)
Softmax over 5 levels	Hugging Face pipeline; scores in `probabilities[]`
Confidence $\max \hat{y}_c$	`confidence = best["probability"]`
Gate if $< \tau$	`bayesian_candidate = confidence < CONFIDENCE_THRESHOLD`
Acuity → SATS colour	`SATS_BY_ACUITY` dict in `predict.py`
TEWS / Bayesian / Fusion	Designed in slides; not in this HTTP predict path (future full pipeline)

When an examiner asks "does the server *implement* fusion?", the honest answer is: **the deployed predict endpoint runs BioBERT inference plus a confidence flag**. Fusion, TEWS, and Bayesian are documented and demonstrated in the presentation; extending the API to run the full `f_fusion` controller would be the next engineering step.

9 Environment variables cheat sheet

Variable	Set on	Example	Purpose
`MODEL_PATH`	ML (local)	path to fine-tuned folder	Load weights from disk
`MODEL_ID`	ML (prod)	`user/curatio-biobert-triage`	Pull from Hub
`ML_SERVICE_URL`	Express	`http://127.0.0.1:8001`	Where to forward predict
`PORT`	Express	5000 (Render injects)	API listen port
`CLIENT_URL`	Express	Vercel URL	CORS allow-list
`CONFIDENCE_THRESHOLD`	ML (+ Express copy)	0.85	Bayesian candidate flag
`VITE_API_URL`	Client build	Render URL	Production API origin

10 What to say in the viva (30-second version)

> *"When a nurse or patient submits a chief complaint, the React client POSTs JSON to our Express API on Render. Express doesn't run the model — it validates the text and forwards the request to a separate Python FastAPI service that loads our fine-tuned BioBERT once at startup. The model tokenises the text to 128 tokens, runs a forward pass through twelve transformer layers, applies softmax over five acuity classes, and returns the predicted level, full probabilities, a SATS colour mapping, and a flag if confidence is below 0.85. That flag corresponds to the Bayesian fallback in our fusion design. Locally, both processes start with one* `npm run dev`*; in production the model weights live on the Hugging Face Hub so we don't bundle 414 megabytes in the API container."*

11 Likely examiner questions

LIKELY EXAMINER QUESTIONS

Q: Why not run BioBERT inside Node.js?
A: The model is PyTorch + Hugging Face `transformers`. Python is the standard runtime for that stack. Node handles HTTP/CORS/deployment ergonomics; Python handles inference. They communicate over a simple REST JSON contract.

Q: What if the ML service crashes?
A: Express catches the connection error and returns HTTP 503 with "ML service unavailable" so the UI can show a clear message instead of hanging.

Q: Is the model loaded on every request?
A: No. `load_model()` uses a module-level singleton (`_pipeline`). FastAPI's lifespan loads it at startup; subsequent requests reuse the same in-memory pipeline.

Q: How do you know the server uses *your* fine-tuned weights?
A: `MODEL_PATH` or `MODEL_ID` points explicitly to the fine-tuned checkpoint (trained on ∼80k merged chief complaints). The `/health` endpoint reports which source was loaded.

Q: Does the API implement TEWS or Bayesian?
A: Not in the current `/predict` endpoint — that runs the BioBERT NLP layer and sets `bayesian_candidate` when confidence is low. TEWS and Bayesian fusion are specified in the system design and presentation; the test page demonstrates the language model path end-to-end.

12 Quick reference: sequence diagram

REQUEST FLOW

1. User clicks **Predict** in browser.
2. `fetch("/api/triage/predict", { text })`
3. Vite proxy (dev) or Vercel → Render (prod)
4. Express `routes/triage.js` validates `text`
5. Express `fetch(ML_SERVICE_URL + "/predict")`
6. FastAPI `predict_endpoint` → `predict(text)`
7. BioBERT pipeline: tokenise → encode → softmax → map acuity/SATS
8. JSON response ↑ Express ↑ browser → UI shows level, colour, bars

---

See also: `curatio/DEPLOYMENT.md` (deploy steps), `presentation-walkthrough.tex` (slide-by-slide defence script). Compile this file with `pdflatex` twice for the table of contents.